

高精度乘单精度

在学习大数乘法前可以先来回顾一下大数加法的计算过程，还是有点相似的图1

$$\begin{array}{r} 123 \\ + 99 \\ \hline 11112 \end{array}$$

Carry values: 1 1 2 (red, purple, blue) and 2 2 2 (red, purple, blue)

图1

从右往左逐位计算，先把每位所计算的结果先写下来，然后再来看哪一位需要发生进位。那么乘法我们知道它是交叉相乘然后再把乘积加起来，如图2所示

$$\begin{array}{r} 21 \\ \times 14 \\ \hline 84 \\ 21 \\ \hline 294 \end{array}$$

图2

现在来把这个竖式乘法换成和大数加法时写的一样呢，用21的每一位去乘以14如图3所示。

$$\begin{array}{r} 21 \\ \times 14 \\ \hline 2814 \end{array}$$

然后再来看需要不需要发生进位，14需要留下4然后向前进1，那么28+9=29如图3所示

$$\begin{array}{r} 21 \\ \times 14 \\ \hline 2814 \\ 29\ 4 \end{array}$$

图3

接着29留下9向前进2，最终结果就是294如图4所示

$$\begin{array}{r}
 21 \\
 \times 14 \\
 \hline
 2814 \\
 294 \\
 294 \\
 \hline
 \end{array}$$

图4

思路差不多一样，也是从右往左逐位开始计算。21用一个字符数组来存放，然后将其转为整形数字倒序存放到整形数组中，而14只需要用一个整形变量存储即可。21当成高精度数，14则是单精度数。**前面有提到过，把21的每一位（2、1）分别和14相乘**，那么倒序也是和大数加法一模一样

```

1 void reverse(char* src, int* arr)
2 {
3     for (int i = 0; i < strlen(src); i++)
4     {
5         arr[i] = src[strlen(src) - 1 - i] - '0';
6     }
7
8 }
9
10 int main()
11 {
12     char s[200] = { 0 };
13     int a[220] = { 0 }; // 高精度数
14     int n = 0; // 单精度数
15
16     scanf("%s %d", s, &n);
17
18
19     // 倒序存放
20     reverse(s, a);
21
22     return 0;
23 }

```

倒序存放数组a后，接着就开始计算乘法了前面说的：**用21的每一位分别和14相乘！**

此时的数组a里存放的是，`a={1,2,0,0,0,0,...}`；那么循环条件应该是 `strlen(s)-1`；如图5

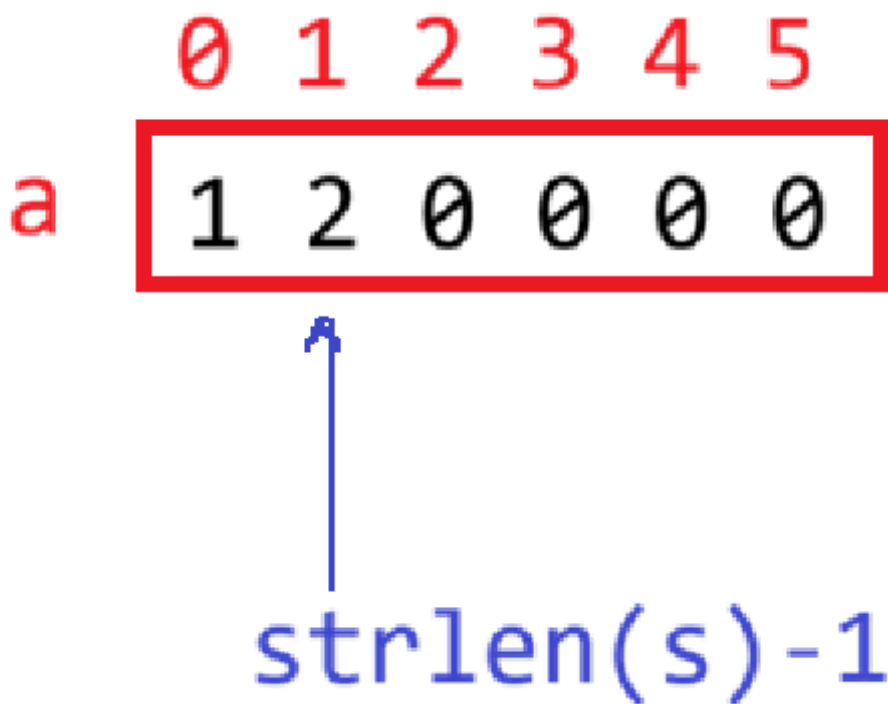


图5

每位相乘的结果都存放在数组 `a` 中，接着就是进位了。先来看看数组 `a` 现在是什么样的如图6所示从 `a[0]` 开始向右遍历去看哪个元素需要进位，如果需要进位则需要留下当前数组的最低位。
`a[1]` 则需要加上进位过来的数字，如14进位后留下4，前进1那么28+进过来的1变成29
来看看结果有多少位，如果单精度数是一个不超过一万的数字那么最大是：9999 那么它的结果是多少位呢？

	0	1	2	3
a	14	28	0	0

$$9 \times 9999 = 89991$$

$$99 \times 9999 = 989901$$

$$999 \times 9999 = 9989001$$

$$9999 \times 9999 = 99980001$$

图6

拿不同的位数去和9999相乘它们的结果：不管是几位数和9999相乘它们的结果都是这个数的位数加上9999的位数。图如6

```

1 void mul(int* arr, int x, size_t size_s)
2 {
3     // 相乘
4     for (int i = 0; i < size_s; i++)
5     {
6         arr[i] = arr[i] * x;
7     }
8
9     // 进位
10    for (int i = 0; i < size_s+4; i++)
11    {
12
13        arr[i + 1] = arr[i + 1] + arr[i] / 10;
14        arr[i] = arr[i] % 10;
15    }
16
17 }
18

```

接着倒序输出数组 `a`，倒序开始从第一个非0的元素开始输出。先来看此时的数组 `a` 里面的元素如图7所示



图7

先找到第一个非0的元素，记录该位置，然后从这个位置开始倒着输出。和大数减法差不多

```

1 // 倒序输出
2 void print(int* arr, size_t len)
3 {
4     // 找到第一个非0元素
5     int index = 0;
6     for (int i = (int)len + 4; i >= 0; i--)
7     {
8         if (arr[i] != 0)
9         {
10             index = i;
11             break;
12         }
13     }
14
15     for (int i = index; i >= 0; i--)
16     {
17         printf("%d", arr[i]);
18     }
19
20 }
```

完整代码：

```

1 #define _CRT_SECURE_NO_WARNINGS 1
2
```

```
3
4 #include <stdio.h>
5 #include <string.h>
6
7 // 转为整数倒序存放
8 void reverse(char* src, int* arr)
9 {
10     for (int i = 0; i < strlen(src); i++)
11     {
12         arr[i] = src[strlen(src) - 1 - i] - '0';
13     }
14
15 }
16
17 // 相乘和进位
18 void mul(int* arr, int x, size_t size_s)
19 {
20     // 相乘
21     for (int i = 0; i < size_s; i++)
22     {
23         arr[i] = arr[i] * x;
24     }
25
26     // 进位
27     for (int i = 0; i < size_s+4; i++)
28     {
29
30         arr[i + 1] = arr[i + 1] + arr[i] / 10;
31         arr[i] = arr[i] % 10;
32     }
33
34 }
35
36 // 倒序输出
37 void print(int* arr, size_t len)
38 {
39     // 找到第一个非0元素
40     int index = 0;
41     for (int i = (int)len + 4; i >= 0; i--)
42     {
43         if (arr[i] != 0)
44         {
45             index = i;
46             break;
47         }
48     }
49
50     for (int i = index; i >= 0; i--)
```

```
51     {
52         printf("%d", arr[i]);
53     }
54
55 }
56
57
58 int main()
59 {
60     char s[200] = { 0 };
61     int a[220] = { 0 }; // 高精度数
62     int n = 0; // 单精度数
63
64
65     scanf("%s %d", s, &n);
66
67     size_t len = strlen(s);
68
69     // 倒序存放
70     reverse(s, a);
71     //每位和n相乘
72     mul(a,n, len);
73
74     // 倒序输出
75     print(a, len);
76
77     return 0;
78 }
```